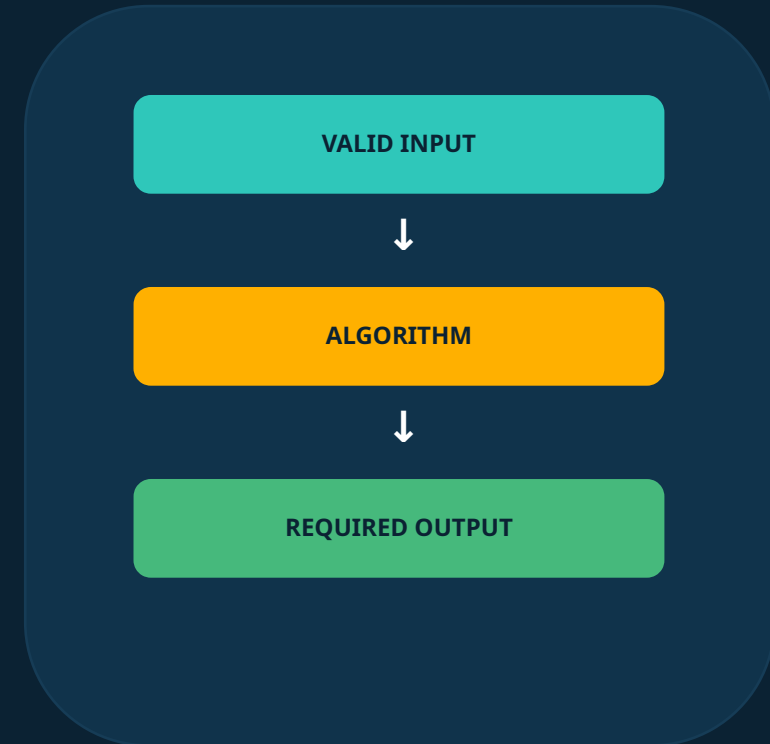


Algorithm Correctness Fundamentals

How do we know an algorithm is actually correct?

From “it seems to work” to showing that it works for every valid input.



**How do we know
this ALWAYS works?**

Working once does not mean correct

A few successful tests can hide a correctness error.

ALGORITHM

MAX-WRONG(A)

largest $\leftarrow 0$

for each x in A
 if $x > \text{largest}$
 largest $\leftarrow x$

return largest

TEST 1

A = [3, 8, 5]
Output = 8 ✓

TEST 2

A = [-8, -3, -12]
Output = 0 ✗
Correct answer = -3

Passing tests $\neq$ proof of correctness

We need a way to reason about every valid input.

Lecture 3 develops the language and reasoning tools needed to make that claim precise.

What will we learn today?

The CDF language is translated here into the questions students should be able to answer.

1 Before an algorithm starts — what must already be true?

PRECONDITIONS

2 After it finishes — what must be true about the result?

POSTCONDITIONS

3 What if the result is right whenever execution finishes?

PARTIAL CORRECTNESS

4 How do we know the algorithm actually finishes?

TERMINATION

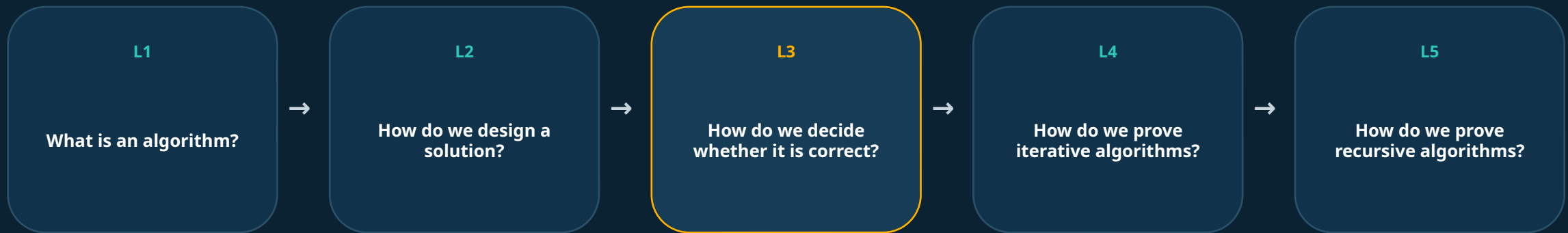
5 When can we call an algorithm completely correct?

TOTAL CORRECTNESS

6 How do we reason about all of this systematically?

MAXIMUM-FINDING EXAMPLE

Where Lecture 3 fits



Today we build the foundation needed for the proofs in Lectures 4 and 5.

Preconditions → Algorithm → Postconditions + Termination

Recall: properties of an algorithm

Lecture 1 introduced the properties. Lecture 3 now zooms in on correctness and finiteness.

INPUT

accepts values from the stated input domain

OUTPUT

produces the result required by the problem

PRECISION

operations are accurately specified

DEFINITENESS

every step has one clear meaning

FINITENESS

eventually terminates

GENERALITY

works for the complete stated input domain

CORRECTNESS

produces the required result for every valid input

Today: CORRECTNESS + FINITENESS → complete correctness

Understanding algorithm properties I

Use MAX2(a,b) to separate four ideas students often mix together.

EXAMPLE

MAX2(a,b)

```
if a ≥ b
    return a
else
    return b
```

INPUT

What comes in?
Here: a and b

OUTPUT

What must come out?
Here: the larger value

PRECISION

Is the operation specified accurately?
Good: if a ≥ b

DEFINITENESS

Can the instruction have only one meaning?
Bad: "return the one that looks bigger"

Precision = accurate operation | Definiteness = unambiguous instruction

Understanding algorithm properties II

These three properties lead directly into formal correctness.

EXAMPLE

```
SUM(n)
total ← 0
for i ← 1 to n
    total ← total + i
return total
```

FINITENESS

Will it stop?

The loop progresses from 1 toward n and eventually ends.

GENERALITY

Does it solve the whole stated problem?

print 5050 solves only $n = 100$.

CORRECTNESS

Does it produce the required result for every valid input?

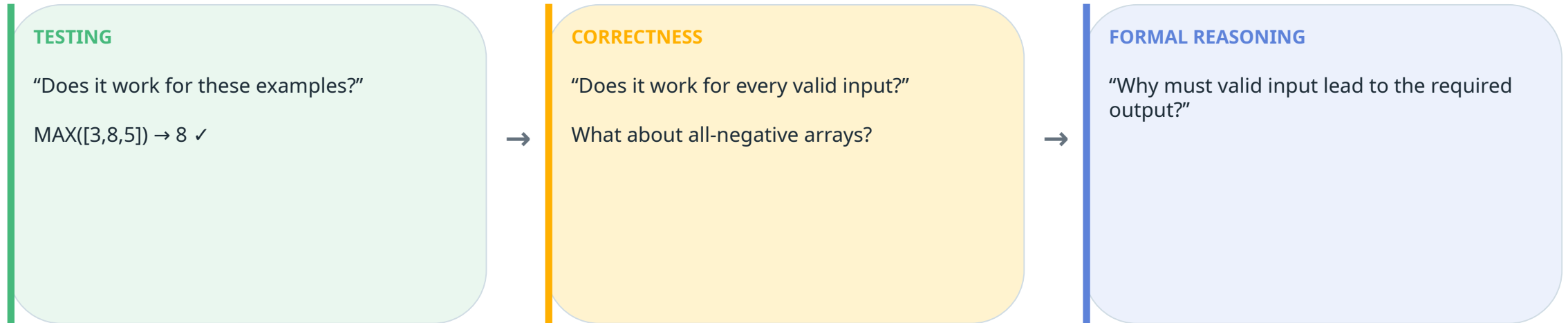
Testing examples alone cannot prove this.

Lecture 1: these are properties → Lecture 3: how do we establish them?

Finiteness will become termination reasoning; correctness will become a formal claim about valid inputs and required outputs.

From “it works” to “it is correct”

Testing gives evidence. Correctness makes a claim about the entire valid input domain.



A few successful tests cannot establish correctness for every valid input.

Correctness starts with two questions

BEFORE the algorithm runs...

What must already be true?



AFTER the algorithm finishes...

What must now be true?

PRECONDITION

POSTCONDITION

A correctness claim connects a valid starting condition to a required finishing condition.

Preconditions — what must be true before we start?

A precondition defines the valid starting state under which the algorithm promises to work.

DIVISION

DIVIDE(a,b)
return a / b

Precondition: $b \neq 0$
English: the divisor must not be zero.

FIRST ELEMENT

FIRST(A)
return A[0]

Precondition: $|A| \geq 1$
English: the array must not be empty.

BINARY SEARCH

BINARY-SEARCH(A,x)

Precondition: A is sorted
English: the algorithm relies on ordering.

A precondition answers: “Under what conditions does this algorithm promise to work?”

If the precondition is false, the correctness guarantee may no longer apply.

Where do preconditions come from?

They are not arbitrary: they come from the problem, the operations, and the assumptions used by the algorithm.

Problem specification

“positive integer n ”

$n \geq 1$

Operation requirement

division

$b \neq 0$

Data structure access

$A[0]$

$|A| \geq 1$

Algorithm assumption

Binary Search

A is sorted

Mathematical domain

real square root

$x \geq 0$

Do not add stronger preconditions than the algorithm really needs.

Example: MAX(A) does not need A to be sorted.

Postconditions — what must be true when we finish?

A postcondition describes the promised result, and sometimes what must remain unchanged.

DIVISION

If the result is q :

$$q = a / b$$

English: q equals a divided by b .

SEARCH

If index i is returned:

$$A[i] = x$$

If NOT-FOUND is returned:

$$x \notin A$$

READ-ONLY MAX

Result property:

$$m = \max(A)$$

Unchanged-state property:

$$A_{\text{after}} = A_{\text{before}}$$

A postcondition answers: “If the algorithm finishes correctly, what must now be true?”

Postconditions can describe both the returned result and properties of the state after execution.

A postcondition must be complete

A condition can be true and still be too weak to describe the required answer.

MAX2 — WEAK CONDITION

Suppose $a = 5$ and $b = 8$.

Condition: $m \geq a$ and $m \geq b$

$m = 100$ satisfies both — but 100 is not the maximum input.

SORT — WEAK CONDITION

Condition: "B is sorted."

$B = []$ is sorted too.

But a sorting algorithm must preserve the input elements as well as order them.

Good postconditions say enough to uniquely capture the intended result.

For MAX2 we also need $m = a$ OR $m = b$.

Mathematical symbols we'll use

Understand the English meaning first; the symbols are only compact shorthand.

 $\forall$

for all / every

 $\forall x \in A$

for every element x in A

 $\in$

is an element of

 $m \in A$

m belongs to A

 $\notin$

is not an element of

 $x \notin A$

x does not occur in A

 $\geq$

greater than or equal to

 $m \geq x$

m is at least as large as x

 $\leq$

less than or equal to

 $m \leq x$

m is no larger than x

 $\neq$

not equal to

 $b \neq 0$

b is not zero

 $\wedge$

and

 $P \wedge Q$

both P and Q are true

 $\vee$

or

 $m=a \vee m=b$

m is either a or b

 $\Rightarrow$

implies

 $P \Rightarrow Q$

if P is true, Q follows

$\forall x \in A, m \geq x = \text{"No element in } A \text{ is larger than } m."$

From precondition to postcondition



If P is true before execution, then when the algorithm finishes we want Q to be true.

DIVIDE: $b \neq 0 \rightarrow q \leftarrow a / b \rightarrow q = a / b$

A correctness claim is always relative to its stated precondition.

But one question remains: what if the algorithm never finishes?

Partial correctness

The result is guaranteed to be right if execution reaches the end.

If the precondition is true AND the algorithm terminates, the postcondition is true.

Key phrase: "IF IT TERMINATES."

WHAT IT TELLS US

The result satisfies the postcondition whenever execution finishes.

WHAT IT DOES NOT TELL US

It does not guarantee that execution will ever finish.

$P \wedge \text{termination} \Rightarrow Q$

Read: if P holds and the algorithm finishes, then Q holds.

Why partial correctness is not enough

An algorithm can return the right answer whenever it finishes and still fail to be completely correct.

EXAMPLE

SQUARE-ONLY-5(x)

if $x = 5$
 return 25

while true
 do nothing

FOR $x = 5$

Returns 25.
That equals x^2 . ✓

FOR $x = 7$

Never returns.
The result is not wrong — there is no result.

Correct result if it stops $\neq$ guaranteed termination

We need a separate termination argument.

Termination — does the algorithm actually finish?

Termination means completing after a finite number of steps for every valid input.

EXAMPLE

COUNTDOWN(n)

```
while n > 0  
  n ← n - 1
```

```
return 0
```

WHY IT FINISHES

Each iteration makes n smaller:
 $5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$
Eventually $n > 0$ becomes false.

WHAT TO LOOK FOR

1. Something changes each iteration.
2. It moves toward a stopping condition.
3. It cannot continue forever.

Termination asks: “Do we definitely finish for every valid input?”

Changing a variable is not enough

The change must move execution toward a state where the loop condition becomes false.

TERMINATES

```
while n > 0  
  n ← n - 1
```

5 → 4 → 3 → 2 → 1 → 0

NEVER CHANGES

```
while n > 0  
  print n
```

5, 5, 5, 5, ...

WRONG DIRECTION

```
while n > 0  
  n ← n + 1
```

5 → 6 → 7 → 8 → ...

Identify what changes, and explain why that change must eventually stop the loop.

Complete / total correctness = right answer + guaranteed termination

PARTIAL CORRECTNESS

+

TERMINATION

=

TOTAL CORRECTNESS

An algorithm is totally correct when it produces the required result AND is guaranteed to terminate for every valid input.

Valid input → correct result → guaranteed finish

Partial correctness vs. total correctness

The difference is entirely about whether termination is guaranteed.

QUESTION	PARTIAL	TOTAL
Correct result if execution finishes?	YES	YES
Termination guaranteed?	NO	YES
Postcondition satisfied?	YES, if it terminates	YES
Completely correct?	Not necessarily	YES

Correct result alone ✗ | Termination alone ✗ | We need both ✓

MAX2 — problem and specification

First complete proof: a simple conditional algorithm with no loop.

ALGORITHM

MAX2(a,b)

```
if  $a \geq b$ 
  return a
else
  return b
```

PRECONDITION

a and b are comparable.
English: the comparison $a \geq b$ must be meaningful.

POSTCONDITION

If the result is m:

$m = a$ OR $m = b$
 $m \geq a$
 $m \geq b$

Therefore $m = \max(a,b)$.

We prove each possible branch satisfies the postcondition.

MAX2 — Case 1: $a \geq b$

Reason from the branch condition to the required postcondition.

BRANCH CONDITION

$a \geq b$

The algorithm executes:
return a

RETURNED VALUE

$m = a$

Therefore m is one of the inputs.

CHECK THE POSTCONDITION

$m \geq a$ because $a \geq a$
 $m \geq b$ because $a \geq b$

So $m = \max(a, b)$.

If $a \geq b$, returning a is correct.

MAX2 — Case 2: $a < b$

The else branch covers the only remaining possibility.

BRANCH CONDITION

The else branch means:
 $a < b$

The algorithm executes:
return b

RETURNED VALUE

$m = b$

Therefore m is one of the inputs.

CHECK THE POSTCONDITION

$m \geq b$ because $b \geq b$
 $m \geq a$ because $b > a$

So $m = \max(a, b)$.

If $a < b$, returning b is correct.

MAX2 — termination and total correctness

PARTIAL CORRECTNESS

Both branches return the correct maximum.

TERMINATION

One comparison → one branch → one return.
No loop. No recursion.

CONCLUSION

Partial correctness + termination = total correctness.

MAX2 is totally correct for inputs satisfying its precondition.

ABS(x) — specify it yourself

A second conditional example before we move to arrays and loops.

ALGORITHM

```
ABS(x)
if  $x \geq 0$ 
  return x
else
  return -x
```

PRECONDITION

$x \in \mathbb{R}$
English: x is a real number.

POSTCONDITION

If the result is r:

$r = |x|$
Equivalent idea:
 $r \geq 0$ and $r = x$ OR $r = -x$

Class question: why does each branch satisfy $r = |x|$?

ABS(x) — correctness reasoning

Same structure as MAX2, but with less scaffolding.

CASE 1: $X \geq 0$

The algorithm returns $r = x$.
Since x is already non-negative,
 $r = |x|$.

CASE 2: $X < 0$

The algorithm returns $r = -x$.
Because x is negative, $-x$ is positive,
so $r = |x|$.

TERMINATION

One comparison, one branch, one return.
Termination is guaranteed.

Both branches satisfy the postcondition + termination is guaranteed → ABS is totally correct.

A systematic correctness checklist

Use this sequence whenever you analyze a new algorithm.

1

STATE

What problem are we solving? What result is required?

2

PRECONDITIONS

What inputs/states are valid before execution?

3

POSTCONDITIONS

What must be true when execution finishes?

4

PARTIAL CORRECTNESS

Assuming valid input and termination, is the result right?

5

TERMINATION

Why must execution eventually stop?

6

CONCLUDE

If both hold, state total correctness.

This is the analysis pattern we will now apply to MAX(A).

Maximum-finding in an array

ALGORITHM

MAX(A)

largest $\leftarrow$ A[0]

for i $\leftarrow$ 1 to length(A)-1
 if A[i] > largest
 largest $\leftarrow$ A[i]

return largest

PROBLEM STATEMENT

Given a finite, non-empty array A of comparable values, return the maximum element.

EXAMPLE

A = [8, 3, 12, 5, 10]
Expected result = 12

MAX(A): what must be true before execution?

The algorithm relies on several input conditions.

NON-EMPTY

$$n \geq 1$$

$A[0]$ must exist.

FINITE

$$n < \infty$$

The loop must have finitely many elements to examine.

COMPARABLE

The operation $A[i] > \text{largest}$ must be meaningful.

VALID INDEXING

Indices $0 \dots n-1$ refer to valid positions in A.

Compact precondition: A is finite, non-empty, and its elements are mutually comparable.

Are all possible restrictions necessary?

A good precondition says what the algorithm truly needs — not more.

DOES A NEED TO BE SORTED?

NO.

[8, 3, 12, 5, 10] is unsorted and MAX still works.

MUST VALUES BE POSITIVE?

NO.

[-8, -3, -12, -5] is valid.
The maximum is -3.

WHAT IS TRULY REQUIRED?

Non-empty + finite + comparable values + valid indexing.

Do not strengthen a precondition merely to make an algorithm easier to justify.

MAX(A): what must be true after execution?

Let m be the returned value.

1. COMES FROM THE INPUT

$$m \in A$$

English: the returned value is one of the array elements.

2. NOTHING IS LARGER

$$\forall x \in A, m \geq x$$

English: no element in A is larger than m .

3. INPUT UNCHANGED

$$A_{\text{after}} = A_{\text{before}}$$

English: MAX reads the array but does not modify it.

Together, conditions 1 and 2 give: $m = \max(A)$

Why “m is at least as large as every element” is not enough

A postcondition can be true and still describe the wrong result.

Suppose:

$A = [3, 8, 5]$

$m = 100$

CHECK THE WEAK CONDITION

$100 \geq 3 \checkmark$

$100 \geq 8 \checkmark$

$100 \geq 5 \checkmark$

So $\forall x \in A, m \geq x$ is true.

But $100 \notin A$, so 100 is not $\max(A)$.

We need BOTH: $m \in A$ AND $\forall x \in A, m \geq x$

The maximum must come from the array, and nothing in the array may be larger.

Trace MAX(A) on one instance

A trace helps us understand behavior, but it is not a general proof.

A = [8, 3, 12, 5, 10]

STEP	COMPARISON	largest AFTER
Start	largest = 8	8
Check 3	3 > 8 ? No	8
Check 12	12 > 8 ? Yes	12
Check 5	5 > 12 ? No	12
Check 10	10 > 12 ? No	12

OUTPUT

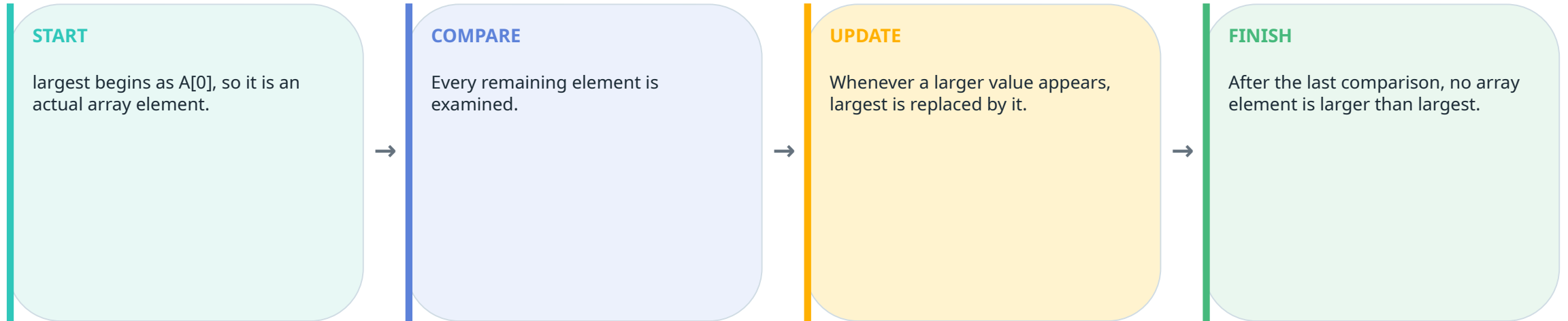
12

Correct for this instance ✓

Question: what would we need to establish before claiming it works for EVERY valid array?

From one trace to general correctness reasoning

Lecture 3 gives the proof idea; Lecture 4 will formalize it with a loop invariant.



At termination: $\text{largest} \in A$ AND $\forall x \in A, \text{largest} \geq x$

So the intended postcondition is reached.

Next lecture: turn this reasoning into a formal loop-invariant proof.

MAX-WRONG(A): can you spot the problem?

Do not reveal the answer immediately — let the class test it.

ALGORITHM

MAX-WRONG(A)

largest $\leftarrow 0$

for each x in A
 if x > largest
 largest $\leftarrow$ x

return largest

TEST 1

A = [3,8,5]
Output = 8 ✓

TEST 2

A = [12,4,9]
Output = 12 ✓

NOW TRY

A = [-8, -3, -12, -5]

Output = 0 ✗ Correct maximum = -3

An algorithm can pass several tests and still be wrong.

Why MAX-WRONG fails

The initialization introduced a hidden assumption that was never part of the problem.

largest $\leftarrow 0$

HIDDEN ASSUMPTION

The algorithm behaves as if $\max(A) \geq 0$.

BUT THE VALID DOMAIN ALLOWS

$A = [-8, -3, -12]$
which has maximum -3 .

Correct initialization:

largest $\leftarrow A[0]$

WHY THIS IS BETTER

largest begins as a real candidate from the array, helping preserve the requirement that the final answer belongs to A.

Do not strengthen the precondition just to rescue a weak algorithm.

Precondition violation vs. algorithm failure

A wrong outcome can happen for two very different reasons.

CASE 1 — INVALID INPUT

```
FIRST(A)  
return A[0]
```

Precondition: $|A| \geq 1$

Call FIRST([]): the precondition is violated, so the correctness guarantee does not apply.

CASE 2 — VALID INPUT, WRONG RESULT

```
MAX-WRONG([-8,-3,-12])
```

The array is finite, non-empty, and comparable.
The legitimate preconditions hold, yet the output is 0 instead of -3 .

The algorithm itself is incorrect.

Wrong result with valid input = correctness failure.

Wrong result outside the stated domain does not automatically prove the algorithm incorrect.

Practice 1 — specify AVERAGE(A)

Identify the preconditions, postcondition, termination argument, and whether the array changes.

ALGORITHM

AVERAGE(A)

sum $\leftarrow$ 0

for each x in A
 sum $\leftarrow$ sum + x

return sum / length(A)

1

1. What must be true before execution?

2

2. What must be true about the result?

3

3. Why does the loop terminate?

4

4. Is A modified?

Discuss for 2–3 minutes before revealing the model answer.

Practice 1 — model answer for AVERAGE(A)

Notice how one small algorithm uses almost every concept from the lecture.

PRECONDITIONS

$|A| \geq 1$
A is finite
Elements are numeric / support + and division

POSTCONDITION

If the result is r:
 $r = (\text{sum of all elements}) / |A|$
Aafter = Abefore

TERMINATION

The loop visits each element once.
Because A is finite, the loop eventually ends.

Why require $|A| \geq 1$? Otherwise $\text{length}(A) = 0$ and the final division is undefined.

Practice 2 — find the hidden error

Problem: given a non-empty numeric array, return its minimum element.

ALGORITHM

```
MIN(A)
smallest  $\leftarrow$  0

for each x in A
  if x < smallest
    smallest  $\leftarrow$  x

return smallest
```

TRY THIS INPUT

$A = [4, 7, 2, 9]$

WHAT HAPPENS?

No element is < 0.
The algorithm returns 0.

WHY IS THAT WRONG?

$0 \notin A$ and $\min(A) = 2$.
The initialization introduced the same hidden assumption as MAX-WRONG.

Correct initialization: smallest $\leftarrow A[0]$

Practice 3 — partial correctness vs. termination

This example separates “right answer if we finish” from “guaranteed to finish.”

ALGORITHM

FIND-ZERO(A)

$i \leftarrow 0$

while $A[i] \neq 0$
 $i \leftarrow i + 1$

return i

CONTAINS ZERO

$A = [5, 7, 3, 0, 9]$
Returns $i = 3$ ✓

NO ZERO

$A = [5, 7, 3, 9]$
The search runs past the array.

KEY QUESTION

If the algorithm stops at i , then $A[i] = 0$ — the result is right.
But what condition is needed to guarantee that it actually stops?

Useful precondition: $0 \in A$

A safer FIND-ZERO algorithm

Better design can remove an unnecessary precondition and handle NOT-FOUND explicitly.

IMPROVED ALGORITHM

FIND-ZERO(A)

$i \leftarrow 0$

while $i < \text{length}(A)$ AND $A[i] \neq 0$
 $i \leftarrow i + 1$

if $i = \text{length}(A)$
 return NOT-FOUND

return i

IF ZERO EXISTS

The algorithm stops at an index i where $A[i] = 0$ and returns i .

IF ZERO DOES NOT EXIST

The loop stops when $i = \text{length}(A)$ and returns NOT-FOUND.

Termination is now guaranteed for every finite A.

Correctness comes before efficiency

CANDIDATE A

Returns the correct answer in 10 seconds.

Correct ✓

CANDIDATE B

Returns in 0.001 seconds...
but sometimes the answer is wrong.

Incorrect ✗

First: Is it correct? → Then: How efficient is it?

Efficiency comparisons are meaningful only among algorithms that correctly solve the same problem.

Lecture 3 recap

Five ideas to carry forward.

01

Preconditions

What must be true before execution.

02

Postconditions

What must be true after execution.

03

Partial correctness

If execution finishes, the result is right.

04

Termination

Execution is guaranteed to finish for every valid input.

05

Total correctness

Partial correctness + termination.

Correctness must be established before efficiency comparisons are meaningful.

Where do we go next?

We reasoned informally about $\text{MAX}(A)$. Now we need a proof method for loops.

INITIALIZATION

Is the property true before the loop starts?

MAINTENANCE

If true before an iteration, does it remain true after?

TERMINATION

When the loop ends, does the property prove the result?

LECTURE 4 — LOOP INVARIANTS AND ITERATIVE CORRECTNESS

Lecture 3 tells us what correctness means. Lecture 4 teaches us how to prove it for loops.